University of Wah

Department of Computer Science

Submit to:

Mr. Hassan Shafaqat

Submit by:

Muhammad Talha

Course:

Information Security Lab

Reg No:

UW-24-CS-BS-101

Section:

BS-CS-3rd-B

Final Lab Sessional
Task # 01

```
def check_discount():
    print("=== UNIVERSITY CAFETERIA DISCOUNT SYSTEM ===")
    print()

    try:
        entry_time = int(input("Enter entry time (24-hour format, e.g., 1400 for 2:00 P
        day_type = input("Enter day type (weekday/weekend): ").lower().strip()
        card_balance = float(input("Enter card balance: $"))
    except ValueError:
        print("Invalid input! Please enter valid values.")
        return

    print("\n" + "="*50)
    print("DECISION PROCESS:")
    print("="*50)

    discount_type = "No Discount"
    reason = []

    morning = (600, 1100)
    peak_hours = (1130, 1330)
    evening = (1700, 2100)

    time_category = ""
    if morning[0] <= entry_time <= morning[1]:
        time_category = "morning"
        print(f"✓ Time: {entry_time} → Morning hours (6:00 AM - 11:00 AM)")
    elif peak_hours[0] <= entry_time <= peak_hours[1]:
        print(f"✓ Time: {entry_time} → Peak hours (11:30 AM - 1:30 PM)")
    elif evening[0] <= entry_time <= evening[1]:
        time_category = "evening"
        print(f"✓ Time: {entry_time} → Evening hours (5:00 PM - 9:00 PM)")
    else:
        time_category = "other"
        print(f"✓ Time: {entry_time} → Outside special hours")


    print(f"✓ Day: {day_type.capitalize()}")

    print(f"✓ Balance: ${card_balance:.2f}")

    print("-" * 50)

    if card_balance < 5.00:
        discount_type = "No Discount"
        reason.append("Balance below $5.00")
        print(" RULE TRIGGERED: Balance below $5.00 → Automatic denial")

    elif day_type == "weekend" and time_category != "peak":
        if card_balance >= 15.00:
            discount_type = "Full Discount"
            reason.append("Weekend with sufficient balance")
            print("✓ RULE TRIGGERED: Weekend (non-peak) with balance ≥ $15.00")
        elif card_balance >= 10.00:
            discount_type = "Partial Discount"
            reason.append("Weekend with moderate balance")
            print("✓ RULE TRIGGERED: Weekend (non-peak) with balance ≥ $10.00")
```

```
elif time_category == "peak":
    if card_balance >= 25.00:
        discount_type = "Partial Discount"
        reason.append("Peak hours with high balance")
        print("✓ RULE TRIGGERED: Peak hours with balance ≥ $25.00")
    else:
        discount_type = "No Discount"
        reason.append("Peak hours - balance insufficient")
        print(" RULE TRIGGERED: Peak hours require balance ≥ $25.00")

elif time_category == "evening":
    if card_balance >= 15.00:
        discount_type = "Partial Discount"
        reason.append("Evening with sufficient balance")
        print("✓ RULE TRIGGERED: Evening hours with balance ≥ $15.00")
    else:
        discount_type = "No Discount"
        reason.append("Evening but balance too low")
        print("RULE TRIGGERED: Evening hours require balance ≥ $15.00")

else:
    if card_balance >= 12.00:
        discount_type = "Partial Discount"
        reason.append("Regular hours with sufficient balance")
        print("✓ RULE TRIGGERED: Regular hours with balance ≥ $12.00")
    else:
        discount_type = "No Discount"
        reason.append("Regular hours - insufficient balance")
        print(" RULE TRIGGERED: Regular hours require balance ≥ $12.00")
```

```
    print("="*50)
    print("\nFINAL DECISION:")
    print("="*50)
    print(f"DISCOUNT STATUS: {discount_type}")
    print(f"REASON: {' | '.join(reason)}")

    if discount_type == "Full Discount":
        print("MEAL PRICE: $0.00 (100% discount applied)")
    elif discount_type == "Partial Discount":
        print("MEAL PRICE: 50% off regular price")
    else:
        print("MEAL PRICE: Regular price (no discount)")

    print("="*50)

if __name__ == "__main__":
    check_discount()
```

Output

```
=== UNIVERSITY CAFETERIA DISCOUNT SYSTEM ===

Enter entry time (24-hour format, e.g., 1400 for 2:00 PM): 3
Enter day type (weekday/weekend): 4
Enter card balance: $200

==================================================
DECISION PROCESS:
==================================================
✓ Time: 3 → Outside special hours
✓ Day: 4
✓ Balance: $200.00
--------------------------------------------------
✓ RULE TRIGGERED: Regular hours with balance ≥ $12.00
==================================================

FINAL DECISION:
==================================================
DISCOUNT STATUS: Partial Discount
REASON: Regular hours with sufficient balance
MEAL PRICE: 50% off regular price
==================================================
```

Task # 02

```
print("TRAFFIC MONITORING SYSTEM")

checkpoints = ["Speed Check", "Signal Jump", "Zone Entry", "License Check", "Document Check"]
max_violations = 3
violations = 0

print(f"Max violations allowed: {max_violations}")
print("-" * 30)

for i, checkpoint in enumerate(checkpoints):
if violations >= max_violations:
print(f"Check {i+1}: {checkpoint} - SKIPPED (Already flagged)")
continue
print(f"Check {i+1}: {checkpoint}")
violation = input("Violation? (yes/no): ").lower()
if violation == "yes":
violations += 1
print(f" Violation added. Total: {violations}")
if violations >= max_violations:
print(" CRITICAL LIMIT REACHED!")

print("\n" + "=" * 30)
print("FINAL REPORT:")
print(f"Checkpoints processed: {len(checkpoints)}")
```

```
print(f"Total violations: {violations}")
if violations >= max_violations:
print("STATUS: FINED - Vehicle impounded")
elif violations > 0:
print("STATUS: WARNING - Fine notice sent")
else:
print("STATUS: CLEAN - No violations")
```

Output

```
TRAFFIC MONITORING SYSTEM
Max violations allowed: 3
------------------------------
Check 1: Speed Check
Violation? (yes/no): tes
Check 2: Signal Jump
Violation? (yes/no): yes
  Violation added. Total: 1
Check 3: Zone Entry
Violation? (yes/no): yes
  Violation added. Total: 2
Check 4: License Check
Violation? (yes/no): no
Check 5: Document Check
Violation? (yes/no): no

==============================
FINAL REPORT:
Checkpoints processed: 5
Total violations: 2
STATUS: WARNING - Fine notice sent
```

Task # 02

```
print("ELECTRICITY USAGE TRACKER")

flats = {}
current_flat = ""

while True:
flat_id = input("\nEnter flat number (or 'done' to finish): ")
if flat_id.lower() == "done":
break
if flat_id not in flats:
flats[flat_id] = []
print(f"New flat {flat_id} added.")
current_flat = flat_id
readings = []
print(f"\nEnter readings for Flat {flat_id} (negative to stop):")
while True:
```

```
try:
reading = float(input("Reading (kWh): "))
if reading < 0:
break
readings.append(reading)
except:
print("Invalid input. Enter number.")
if readings:
avg = sum(readings) / len(readings)
max_reading = max(readings)
flats[flat_id] = [readings, avg, max_reading]

print("\n" + "=" * 40)
print("MONTHLY REPORT")
print("=" * 40)

most_inconsistent = ""
highest_variation = 0

for flat_id, data in flats.items():
readings, avg, peak = data
if readings:
variation = max(readings) - min(readings)
print(f"\nFlat {flat_id}:")
print(f" Readings: {len(readings)}")
print(f" Average: {avg:.1f} kWh")
print(f" Peak: {peak:.1f} kWh")
print(f" Variation: {variation:.1f} kWh")
if variation > highest_variation:
highest_variation = variation
most_inconsistent = flat_id

if flats:
all_readings = []
for flat_id, data in flats.items():
all_readings.extend(data[0])
if all_readings:
print("\n" + "=" * 40)
print("BUILDING SUMMARY:")
print(f"Total flats: {len(flats)}")
print(f"Total readings: {len(all_readings)}")
print(f"Building average: {sum(all_readings)/len(all_readings):.1f} kWh")
print(f"Building peak: {max(all_readings):.1f} kWh")
print(f"Most inconsistent: Flat {most_inconsistent}")
```

Output

```
ELECTRICITY USAGE TRACKER

Enter flat number (or 'done' to finish): 2
New flat 2 added.

Enter readings for Flat 2 (negative to stop):
Reading (kWh): 311
Reading (kWh): 312
Reading (kWh): -2

Enter flat number (or 'done' to finish): done

========================================
MONTHLY REPORT
========================================

Flat 2:
  Readings: 2
  Average: 311.5 kWh
  Peak: 312.0 kWh
  Variation: 1.0 kWh

========================================
BUILDING SUMMARY:
Total flats: 1
Total readings: 2
Building average: 311.5 kWh
Building peak: 312.0 kWh
Most inconsistent: Flat 2
```